Tema 027 – RC: Înscrierea și retragerea dintr-un grup care primește un semnal

File-Mitita Traian · Cisco Packet Tracer 9.0.1, modulul Programming, Python 3

1. Topologia

Cinci plăci SBC-PT (A, B, C, D, E) legate la un switch 2960, toate în subrețeaua 192.168.1.0/24. În Packet Tracer 9 placa SBC vine doar cu Wireless și Bluetooth, așa că pe fiecare am pus modulul **PT-IOT-NM-1CFE** (un port FastEthernet) în tabul Physical și am legat portul `FastEthernet0` la switch.

Placă	Rol	Adresă IPv4	Port switch
A	sursa semnalului, ține lista membrilor	192.168.1.1/24	Fa0/1
B	membru de la început	192.168.1.2/24	Fa0/2
C	membru, se retrage în timpul rulării	192.168.1.3/24	Fa0/3
D	membru de la început	192.168.1.4/24	Fa0/4
E	se înscrie după începerea fluxului	192.168.1.5/24	Fa0/5

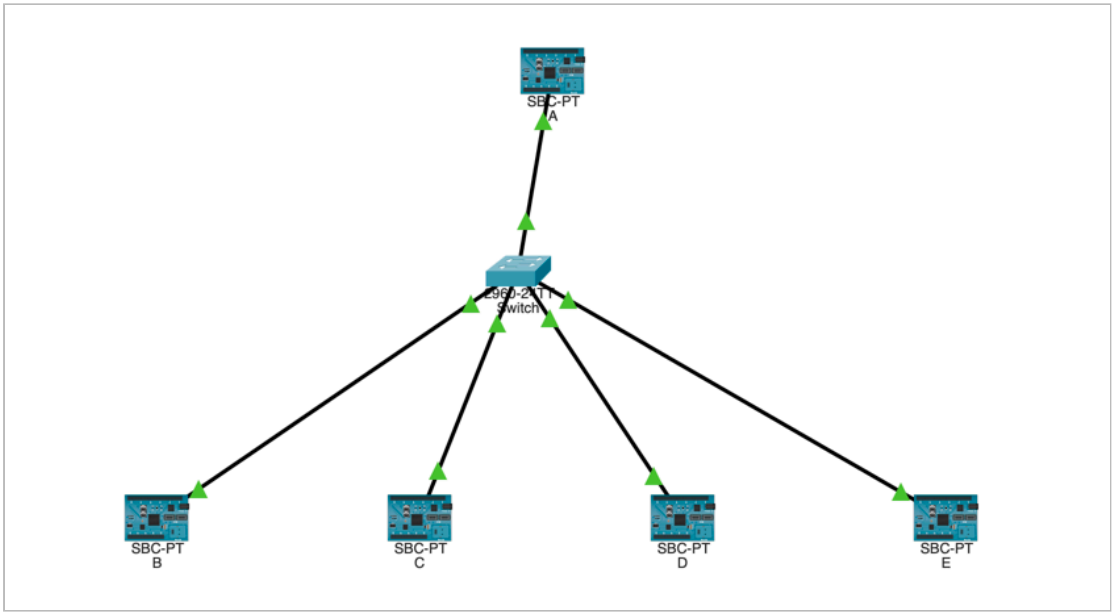


Fig. 1 – Topologia în Packet Tracer: toate legăturile sunt active (verde). Fișierul salvat este Tema027_RC_FileMitita_Traian.pkt.

2. Protocolul la nivel de aplicație

Toate mesajele sunt text, în datagrame UDP pe portul 5000. Distribuția către grup se face prin **câte o datagramă unicast pentru fiecare membru** din lista lui A.

Mesaj	De la → la	Conținut
`DATA;n;x`	A → fiecare membru	indicele eșantionului n și valoarea x[n] = A_s · sin(2π·n / (T·fs))
`JOIN` / `LEAVE`	membru → A	cerere de înscriere / retragere
`ACK;JOIN;k` / `ACK;LEAVE;k`	A → membru	confirmarea schimbării; k = indicele următorului eșantion la A în acel moment
`END;80`	A → membri	sfârșitul fluxului

- Semnalul: A_s = 10, T = 4 s, fs = 2 eșantioane/s, deci un eșantion la 0,5 s și 8 eșantioane pe perioadă; A trimite 80 de eșantioane (40 s).
- **A nu suspendă generarea**: bucla din `main()` generează și trimite eșantioanele, iar cererile JOIN/LEAVE sunt tratate în callback-ul `onUDPReceive`, care doar modifică lista și trimite confirmarea. Pentru fiecare eșantion, A trimite către o copie a listei din acel moment.
- Membrii repetă JOIN (sau LEAVE) la 1 s până primesc ACK. B, C și D cer înscrierea imediat; **E** cere înscrierea la 15 s după pornire, iar **C** se retrage la 45 s după pornire. Fiecare membru notează primul și ultimul indice primit.

3. Rularea

Am pornit programele în ordinea B, C, D, E (la câte ~8 s) și apoi A, ca să existe membri de la începutul fluxului: B 11:21:57, C 11:22:05, D 11:22:13, E 11:22:20, A 11:22:28. Așa, cererea lui E (11:22:35) vine după începerea fluxului, iar retragerea lui C (11:22:50) vine în timpul rulării.

Rezultate la cele două evenimente

Eveniment	Indicele următor la A în momentul schimbării	Indicele observat la membru	Concluzie
E se înscrie (JOIN)	16	primul primit de E: **16**	E primește exact de la eșantionul următor
C se retrage (LEAVE)	46	ultimul primit de C: **45**	C nu mai primește nimic după cerere (0 eșantioane în plus)

Eșantioane primite de fiecare membru

Membru	Primul indice	Ultimul indice	Eșantioane primite
B	1	79	79
C	1	45	45
D	2	79	78
E	16	79	64

Variația numărului de trimiteri per eșantion (la A)

Eșantioane	Membri în listă	Trimiteri per eșantion	Nr. eșantioane
n = 0	–	0	1
n = 1	C, B	2	1
n = 2 … 15	C, B, D	3	14
n = 16 … 45	C, B, D, E	4	30
n = 46 … 79	B, D, E	3	34
Total datagrame DATA			**266**

Verificare: 0 + 2 + 3·14 + 4·30 + 3·34 = 266, același total afișat de A la final.

Interpretare

- Indicele „următor” notat de A la JOIN este chiar primul eșantion primit de E (16), deci înscrierea are efect de la următorul eșantion generat, fără eșantioane pierdute sau primite în plus.
- La LEAVE, ultimul eșantion primit de C este 45 = 46 − 1: după tratarea cererii, A nu mai trimite către C. Nicio datagramă DATA nu era „pe drum” spre C în momentul retragerii (0 eșantioane primite după LEAVE).
- Numărul de trimiteri per eșantion urmează numărul de membri: 3 → 4 după înscrierea lui E → 3 după retragerea lui C. Pentru că distribuția e prin unicast, traficul lui A crește liniar cu numărul de membri.
- Generarea nu s-a oprit în timpul cererilor: eșantioanele au rămas la ~0,5 s unul de altul (coloana t din ieșirea lui A).

4. Capturi din simulare

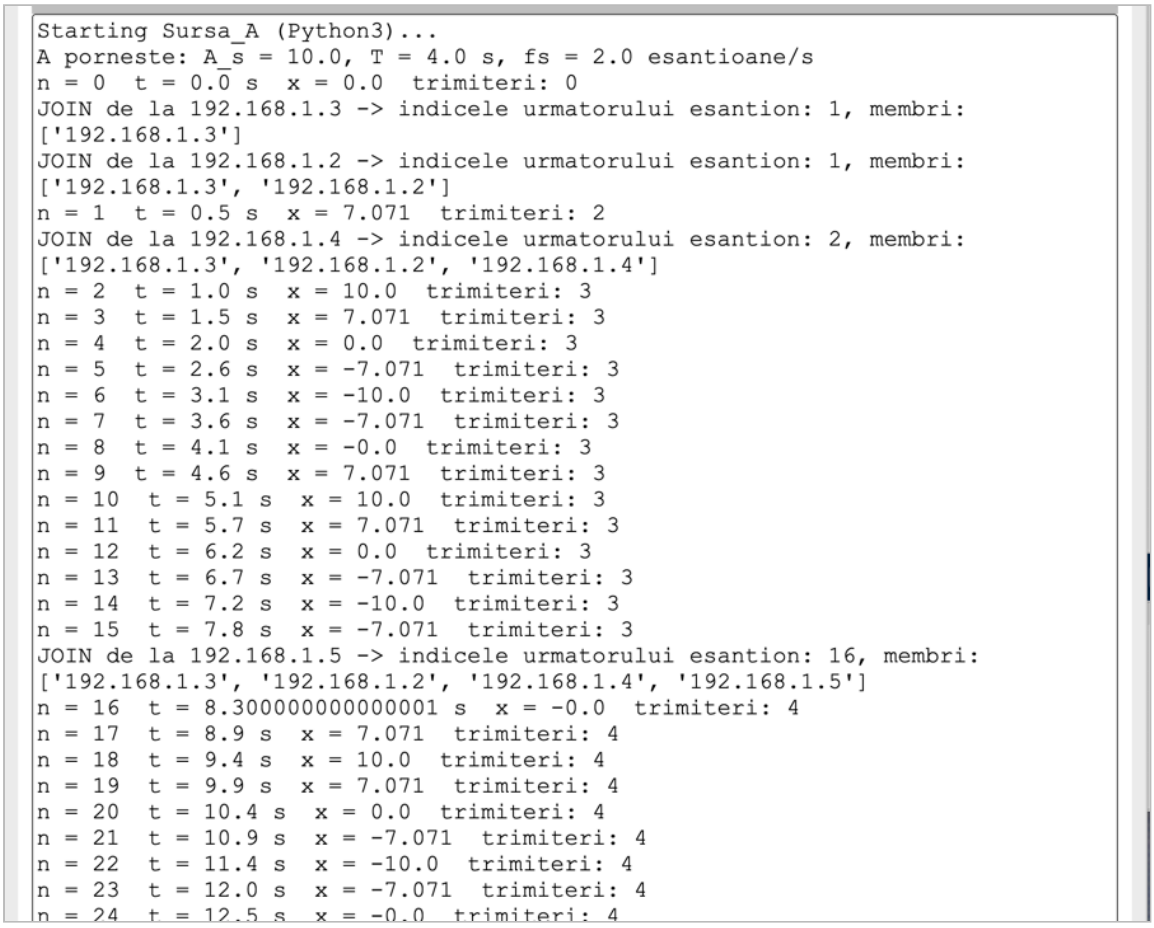


Fig. 2 – Rularea de referință la A: pornire fără membri (n = 0, 0 trimiteri), apoi JOIN de la C, B, D (indicii următori 1, 1, 2) și 3 trimiteri per eșantion. La n = 16 apare JOIN-ul lui E, după care fiecare eșantion se trimite de 4 ori.

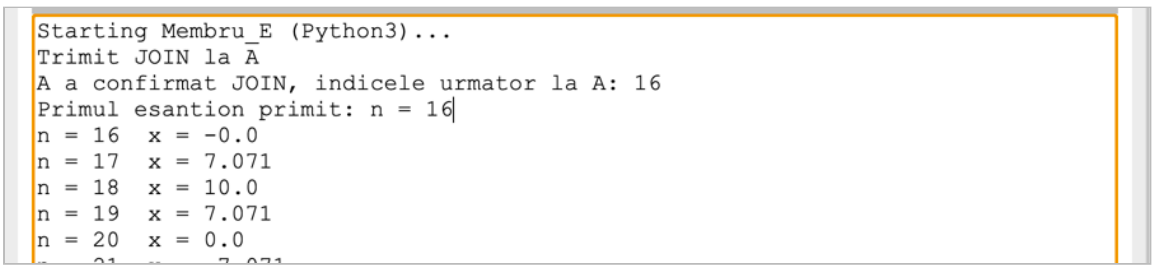


Fig. 3 – La E: cererea JOIN, confirmarea cu indicele următor 16 și primul eșantion primit, n = 16 (același indice).

```
n = 37  t = 19.2 s  x = -7.071  trimiteri: 4
n = 38  t = 19.7 s  x = -10.0  trimiteri: 4
n = 39  t = 20.3 s  x = -7.071  trimiteri: 4
n = 40  t = 20.9 s  x = -0.0  trimiteri: 4
n = 41  t = 21.4 s  x = 7.071  trimiteri: 4
n = 42  t = 21.9 s  x = 10.0  trimiteri: 4
n = 43  t = 22.4 s  x = 7.071  trimiteri: 4
n = 44  t = 22.9 s  x = 0.0  trimiteri: 4
n = 45  t = 23.5 s  x = -7.071  trimiteri: 4
LEAVE de la 192.168.1.3 -> indicele urmatorului esantion: 46, membri:
['192.168.1.2', '192.168.1.4', '192.168.1.5']
n = 46  t = 24.0 s  x = -10.0  trimiteri: 3
n = 47  t = 24.5 s  x = -7.071  trimiteri: 3
n = 48  t = 25.0 s  x = -0.0  trimiteri: 3
n = 49  t = 25.5 s  x = 7.071  trimiteri: 3
```

Fig. 4 – La A: LEAVE de la 192.168.1.3 (C) când indicele următor era 46; de la n = 46 se trimite doar către B, D, E (3 trimiteri).

```
n = 39  x = -7.071
n = 40  x = -0.0
n = 41  x = 7.071
n = 42  x = 10.0
n = 43  x = 7.071
n = 44  x = 0.0
n = 45  x = -7.071
Trimit LEAVE la A
A a confirmat LEAVE, indicele urmator la A: 46
Esantioane primite dupa trimiterea LEAVE: 0
Primul indice primit: 1, ultimul: 45, primite: 45
```

Fig. 5 – La C: ultimul eșantion primit este n = 45, confirmarea LEAVE are indicele 46, iar după LEAVE nu mai sosește nimic.

```
n = 79  t = 41.1 s  x = -7.071  trimiteri: 3
Gata, 80 esantioane. Numarul de trimiteri per esantion:
  0 trimiteri: 1 esantioane
  2 trimiteri: 1 esantioane
  3 trimiteri: 48 esantioane
  4 trimiteri: 30 esantioane
Total datagrame DATA: 266
```

Fig. 6 – Rezumatul de la A: numărul de eșantioane după numărul de trimiteri și totalul de 266 de datagrame DATA.

```
Fluxul s-a terminat la A (80 esantioane)
Primul indice primit: 1, ultimul: 79, primite: 79
```

```
Fluxul s-a terminat la A (80 esantioane)
Primul indice primit: 2, ultimul: 79, primite: 78
```

```
Fluxul s-a terminat la A (80 esantioane)
Primul indice primit: 16, ultimul: 79, primite: 64
```

Fig. 7 – Rezumatele lui B, D și E după mesajul END: primul și ultimul indice primit și numărul de eșantioane.

5. Codul plăcilor

Placa A (proiectul Sursa_A, main.py):

```
# Placa A – sursa semnalului si a listei de membri (Tema 027 RC)
from udp import *
from time import *
import math

PORT = 5000
A_S = 10.0      # amplitudinea sinusoidei
T = 4.0         # perioada, in secunde
FS = 2.0        # rata: esantioane pe secunda
```

```
N_MAX = 80      # cate esantioane genereaza A (40 s)

udp = UDPSocket()
membri = []         # IP-urile membrilor grupului
n = 0               # indicele urmatorului esantion
trimiteri = []      # cate datagrame s-au trimis pentru fiecare esantion


def onUDPReceive(ip, port, data):
    # cererile se trateaza aici, generarea din main() nu se opreste
    if data == "JOIN":
        if ip not in membri:
            membri.append(ip)
            print("JOIN de la " + ip + " -> indicele urmatorului esantion: " + str(n) + ", membri:
" + str(membri))
        udp.send(ip, PORT, "ACK;JOIN;" + str(n))
    elif data == "LEAVE":
        if ip in membri:
            membri.remove(ip)
            print("LEAVE de la " + ip + " -> indicele urmatorului esantion: " + str(n) + ",
membri: " + str(membri))
        udp.send(ip, PORT, "ACK;LEAVE;" + str(n))


def main():
    global n
    udp.onReceive(onUDPReceive)
    udp.begin(PORT)
    print("A porneste: A_s = " + str(A_S) + ", T = " + str(T) + " s, fs = " + str(FS) + "
esantioane/s")
    start = uptime()
    while n < N_MAX:
        x = A_S * math.sin(2 * math.pi * n / (T * FS))
        mesaj = "DATA;" + str(n) + ";" + str(round(x, 3))
        destinatari = list(membri)
        for ip in destinatari:
            udp.send(ip, PORT, mesaj)          # cate o datagrama unicast pentru fiecare membru
        trimiteri.append(len(destinatari))
        print("n = " + str(n) + "  t = " + str(round(uptime() - start, 1)) + " s  x = " +
str(round(x, 3)) + "  trimiteri: " + str(len(destinatari)))
        n = n + 1
        sleep(1.0 / FS)                        # urmatorul esantion dupa 1/fs

    for ip in membri:
        udp.send(ip, PORT, "END;" + str(n))
    print("Gata, " + str(n) + " esantioane. Numarul de trimiteri per esantion:")
    for k in range(0, 6):
        cate = trimiteri.count(k)
        if cate > 0:
            print("  " + str(k) + " trimiteri: " + str(cate) + " esantioane")
    print("Total datagrame DATA: " + str(sum(trimiteri)))
    while True:
        sleep(1)


if __name__ == "__main__":
    main()
```

Plăcile B, C, D, E (proiectele Membru_B … Membru_E, main.py). Codul e același, se schimbă doar `JOIN_DUPA` și `LEAVE_DUPA`: B, D: 0 și −1; C: 0 și 45; E: 15 și −1.

```
# Placile B, C, D, E - membrii grupului (Tema 027 RC)
# Se schimba doar JOIN_DUPA si LEAVE_DUPA:
#   B, D: JOIN_DUPA = 0,  LEAVE_DUPA = -1
#   C:    JOIN_DUPA = 0,  LEAVE_DUPA = 45   (se retrage in timpul rularii)
#   E:    JOIN_DUPA = 15, LEAVE_DUPA = -1   (se inscrie dupa inceperea fluxului)
from udp import *
from time import *

IP_A = "192.168.1.1"
```

```
PORT = 5000
JOIN_DUPA = 0       # secunde de la pornirea programului pana la JOIN
LEAVE_DUPA = -1     # secunde pana la LEAVE (-1 = nu se retrage)

udp = UDPSocket()
stare = "IN_AFARA"  # IN_AFARA -> IN_GRUP -> RETRAS
primul = -1
ultimul = -1
primite = 0
dupa_leave = 0      # esantioane sosite dupa trimiterea LEAVE
leave_trimis = False


def rezumat():
    print("Primul indice primit: " + str(primul) + ", ultimul: " + str(ultimul) + ", primite: " +
str(primite))


def onUDPReceive(ip, port, data):
    global stare, primul, ultimul, primite, dupa_leave
    parti = data.split(";")
    if parti[0] == "DATA":
        idx = int(parti[1])
        if primul < 0:
            primul = idx
            print("Primul esantion primit: n = " + str(idx))
        ultimul = idx
        primite = primite + 1
        if leave_trimis:
            dupa_leave = dupa_leave + 1
        print("n = " + parti[1] + "  x = " + parti[2])
    elif parti[0] == "ACK" and parti[1] == "JOIN" and stare == "IN_AFARA":
        stare = "IN_GRUP"
        print("A a confirmat JOIN, indicele urmator la A: " + parti[2])
    elif parti[0] == "ACK" and parti[1] == "LEAVE" and stare == "IN_GRUP":
        stare = "RETRAS"
        print("A a confirmat LEAVE, indicele urmator la A: " + parti[2])
        print("Esantioane primite dupa trimiterea LEAVE: " + str(dupa_leave))
        rezumat()
    elif parti[0] == "END":
        print("Fluxul s-a terminat la A (" + parti[1] + " esantioane)")
        rezumat()


start = 0


def main():
    global start, leave_trimis
    udp.onReceive(onUDPReceive)
    udp.begin(PORT)
    start = uptime()
    ultima_cerere = -10
    while True:
        t = uptime() - start
        # cererile se repeta la 1 s pana vine confirmarea
        if stare == "IN_AFARA" and t >= JOIN_DUPA and t - ultima_cerere >= 1:
            udp.send(IP_A, PORT, "JOIN")
            ultima_cerere = t
            print("Trimit JOIN la A")
        if stare == "IN_GRUP" and LEAVE_DUPA >= 0 and t >= LEAVE_DUPA and t - ultima_cerere >= 1:
            udp.send(IP_A, PORT, "LEAVE")
            leave_trimis = True
            ultima_cerere = t
            print("Trimit LEAVE la A")
        sleep(0.2)


if __name__ == "__main__":
    main()
```